

CONSERVATOIRE NATIONAL DES ARTS ET METIERS
PARIS

NFE204 – Bases de données documentaires et distribuées

Année 2015

Étude du système NoSQL Apache Cassandra



Hubert Bossot
IDF078-181

Sommaire

Introduction.....	3
Collecte des données.....	4
Insertion dans la base.....	5
Lancement.....	5
Helenos.....	7
Cassandra-Cluster-Admin.....	8
OpsCenter	8
Langage de manipulation de données.....	9
ETL.....	11
Performances.....	16
Conclusion.....	18
Webographie.....	19

Introduction

Les systèmes NoSQL sont de plus en plus employés proportionnellement aux systèmes de gestion de base de données relationnels. En effet, les accès ne nécessitant pas d'aspect transactionnel (ou propriétés ACID) sont plus rapides en l'absence de jointures et de tests d'intégrité.

Rank			DBMS	Database Model	Score		
Jul 2015	Jun 2015	Jul 2014			Jul 2015	Jun 2015	Jul 2014
1.	1.	1.	Oracle	Relational DBMS	1456.72	-9.64	-28.39
2.	2.	2.	MySQL	Relational DBMS	1283.33	+4.98	-12.45
3.	3.	3.	Microsoft SQL Server	Relational DBMS	1103.05	-15.00	-143.54
4.	↑ 5.	↑ 5.	MongoDB +	Document store	287.40	+8.34	+48.62
5.	↓ 4.	↓ 4.	PostgreSQL	Relational DBMS	272.83	-8.08	+33.37
6.	6.	6.	DB2	Relational DBMS	198.12	-0.58	-3.90
7.	7.	7.	Microsoft Access	Relational DBMS	144.30	-2.19	-0.32
8.	8.	↑ 10.	Cassandra +	Wide column store	112.71	+3.80	+31.13
9.	9.	↓ 8.	SQLite	Relational DBMS	105.87	-2.10	+14.72
10.	10.	↑ 12.	Redis +	Key-value store	95.08	-0.41	+26.34
11.	11.	↓ 9.	SAP Adaptive Server	Relational DBMS	87.22	-0.96	+3.49
12.	12.	↓ 11.	Solr	Search engine	79.29	-1.95	+7.77
13.	13.	13.	Teradata	Relational DBMS	72.32	-0.77	+5.85

Figure I : SGBD les plus utilisés dans le monde en juillet 2015

<http://db-engines.com/en/ranking>



Figure II : utilisateurs de Cassandra

<http://cassandra.apache.org/>

Apache Cassandra est un SGBD orienté colonne, distribué, scalable, libre, dont on peut trouver les sources Java sur <https://github.com/apache/cassandra>, mais la société DataStax en fournit une version Enterprise pour Linux et MacOS.

Il n'offre pas de redondance entre ses nœuds, les données perdues en cas de panne sont donc partagées entre les nœuds restant, selon la clé primaire. Il n'y a pas master et donc pas de SPOF, la reprise sur panne s'effectue donc automatiquement : si un nœud est injoignable on essaie de se connecter à un autre et programme d'autres reconnections en parallèle.

Son langage de requête s'appelle CQL et ressemble à SQL.

Il possède un ETL capable d'importer des fichiers CSV.

Collecte des données

La société Narboni fondée en 1963 imprime et découpe dans son siège situé aux Ulis dans l'Essonne, grâce à une centaine de salariés, des solutions sur mesure, essentiellement des cartes de fidélité, cartes cadeaux et produits promotionnels tels que des jaquettes DVD. Elle respecte la NF ISO 9001, et ses labels « Imprim'Vert » et « Imprime luxe ».

En janvier 2015, Narboni remporta l'appel d'offre des cartes cadeaux de l'enseigne Leclerc, grâce notamment à la mise en place d'un site web de suivi des commandes prévu pour mars 2015.

En mars 2015, l'équipe réseau du groupe Taylor refusa d'ouvrir le port 80 du routeur WAN vers le serveur du LAN contenant les commandes et transactions des cartes cadeaux imprimées par Narboni, m'invitant à synchroniser ces dernières avec une plateforme externe.

J'ai donc mis en place un serveur Linux hébergé chez OVH, avec un site réalisé en PHP et MySQL pour permettre à nos clients de suivre leurs commandes de cartes.

Les données sont synchronisées par batch, plusieurs fois par jour, sous forme de fichiers CSV.

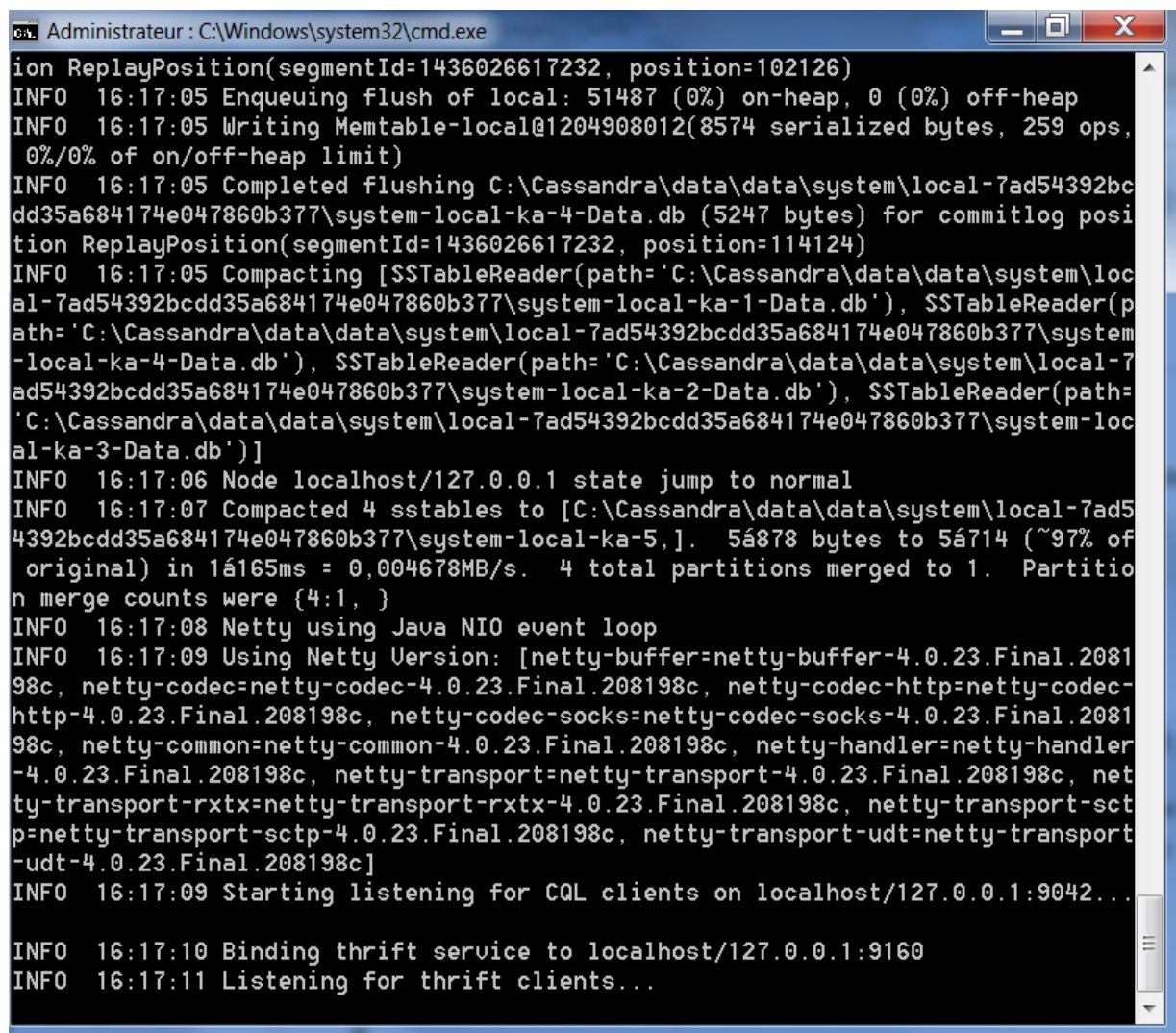
Problème : en juin 2015, le client Banque-Edel qui imprime les cartes Leclerc, avait déjà passé plus de 5 000 commandes, alors même que la plateforme n'était pas dimensionnée pour de mille. Le temps d'affichage du premier élément en AJAX est donc passé à 14 s, et il faut compter 57 s pour assister à la fin du chargement de la liste des 5245 commandes avec leurs 8633 lignes de commandes. Pourtant la base de données ne pèse que 2,38 Mo, et a déjà été optimisée et indexée de différentes manières.

La présente étude a donc pour but de tester les performances du logiciel Apache Cassandra afin d'envisager son utilisation à la place de MySQL pour publier ces données privées.

Insertion dans la base

Lancement

Une fois téléchargé, Cassandra s'exécute sous Windows avec
C:\Cassandra\bin\cassandra.bat :



```
ion ReplayPosition(segmentId=1436026617232, position=102126)
INFO 16:17:05 Enqueuing flush of local: 51487 (0%) on-heap, 0 (0%) off-heap
INFO 16:17:05 Writing Memtable-local@1204908012(8574 serialized bytes, 259 ops,
0%/0% of on/off-heap limit)
INFO 16:17:05 Completed flushing C:\Cassandra\data\data\system\local-7ad54392bcd
dd35a684174e047860b377\system-local-ka-4-Data.db (5247 bytes) for commitlog posi
tion ReplayPosition(segmentId=1436026617232, position=114124)
INFO 16:17:05 Compacting [SSTableReader(path='C:\Cassandra\data\data\system\loc
al-7ad54392bcd35a684174e047860b377\system-local-ka-1-Data.db'), SSTableReader(p
ath='C:\Cassandra\data\data\system\local-7ad54392bcd35a684174e047860b377\system
-local-ka-4-Data.db'), SSTableReader(path='C:\Cassandra\data\data\system\local-7
ad54392bcd35a684174e047860b377\system-local-ka-2-Data.db'), SSTableReader(path=
'C:\Cassandra\data\data\system\local-7ad54392bcd35a684174e047860b377\system-loc
al-ka-3-Data.db')]
INFO 16:17:06 Node localhost/127.0.0.1 state jump to normal
INFO 16:17:07 Compacted 4 sstables to [C:\Cassandra\data\data\system\local-7ad5
4392bcd35a684174e047860b377\system-local-ka-5,]. 56878 bytes to 56714 (~97% of
original) in 16165ms = 0.004678MB/s. 4 total partitions merged to 1. Partitio
n merge counts were {4:1, }
INFO 16:17:08 Netty using Java NIO event loop
INFO 16:17:09 Using Netty Version: [netty-buffer=netty-buffer-4.0.23.Final.2081
98c, netty-codec=netty-codec-4.0.23.Final.208198c, netty-codec-http=netty-codec-
http-4.0.23.Final.208198c, netty-codec-socks=netty-codec-socks-4.0.23.Final.2081
98c, netty-common=netty-common-4.0.23.Final.208198c, netty-handler=netty-handler
-4.0.23.Final.208198c, netty-transport=netty-transport-4.0.23.Final.208198c, net
ty-transport-rxtx=netty-transport-rxtx-4.0.23.Final.208198c, netty-transport-sct
p=netty-transport-sctp-4.0.23.Final.208198c, netty-transport-udt=netty-transport
-udt-4.0.23.Final.208198c]
INFO 16:17:09 Starting listening for CQL clients on localhost/127.0.0.1:9042...
INFO 16:17:10 Binding thrift service to localhost/127.0.0.1:9160
INFO 16:17:11 Listening for thrift clients...
```

Apparemment, il est impossible de lancer deux nœuds sur la même machine : le second se ferme automatiquement, même si les IP de leurs fichiers de configuration C:\cassandra\conf\cassandra.yaml sont différentes.

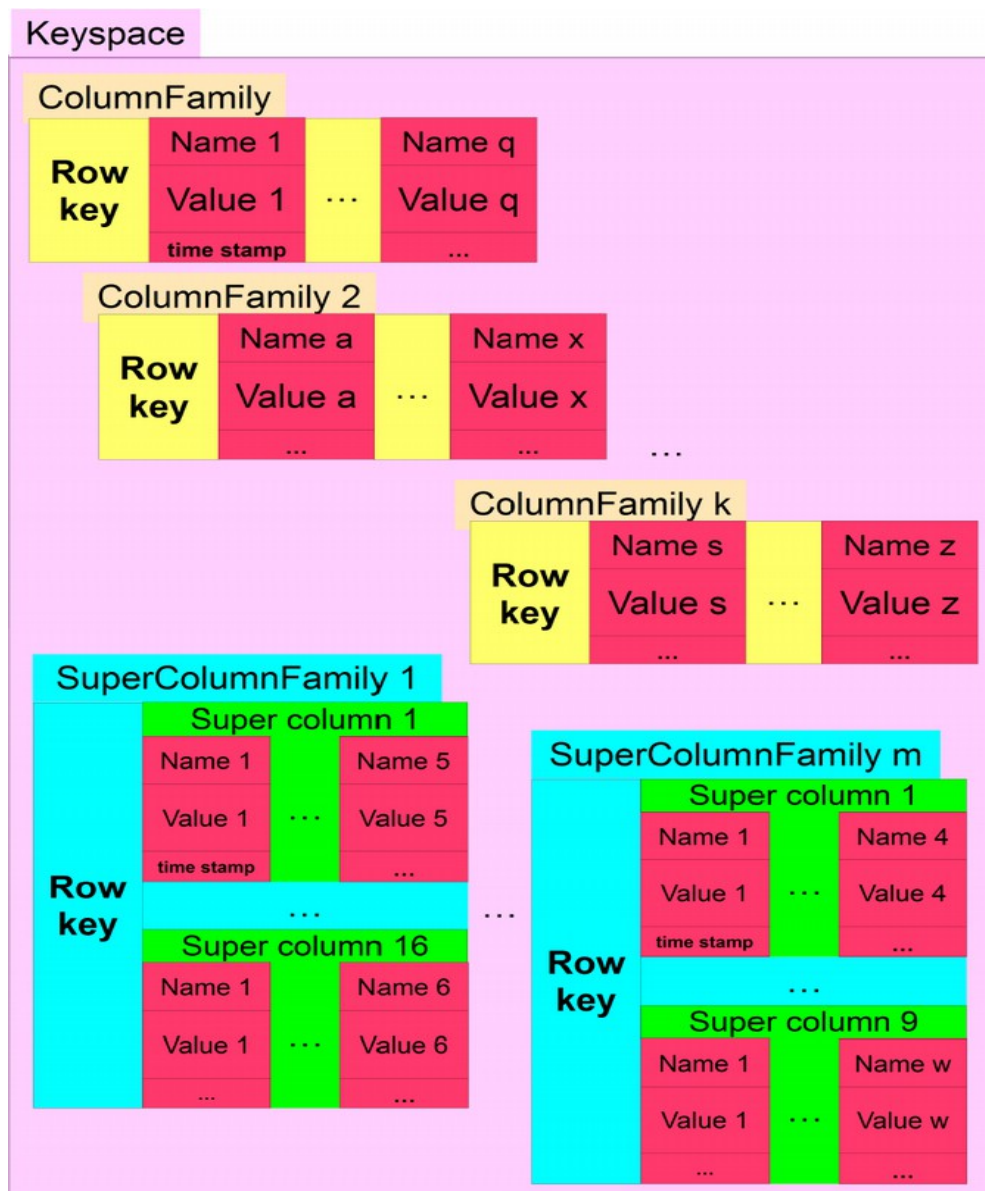


Figure IV : architecture de Cassandra

https://en.wikipedia.org/wiki/File:Keyspace_example_%28data_store%29.png

Une base de données Cassandra stockée en mémoire vive l'est dans l'objet Memtable, si elle dépasse la taille du matériel alors le supplément est inscrit dans l'objet SSTable.

Pour des raisons ergonomiques, on cherche à utiliser de préférence une interface graphique web, plutôt que l'invite de commande `\Cassandra\bin\cqsh.bat`.

La configuration de bases à plusieurs nœuds est décrite sur <http://docs.datastax.com/en/cassandra/2.0/cassandra/initialize/initializeSingleDS.html>.

Elle demande d'ouvrir les ports suivants :

Cassandra inter-node ports

Port number	Description
7000	Cassandra inter-node cluster communication.
7001	Cassandra SSL inter-node cluster communication.
7199	Cassandra JMX monitoring port.

Cassandra client ports

Port number	Description
9042	Cassandra client port.
9160	Cassandra client port (Thrift).

Helenos

Les sources Java de cette interface graphique pour Cassandra sont disponibles sur <https://github.com/tomekkup/helenos> et la version compilée sur <http://sourceforge.net/projects/helenos-gui/>. Elle comprend un serveur Apache + Tomcat. Une fois lancée par `\helenos\bin\startup.bat`, elle est accessible sur <http://localhost:8080> (identifiants admin / admin) :

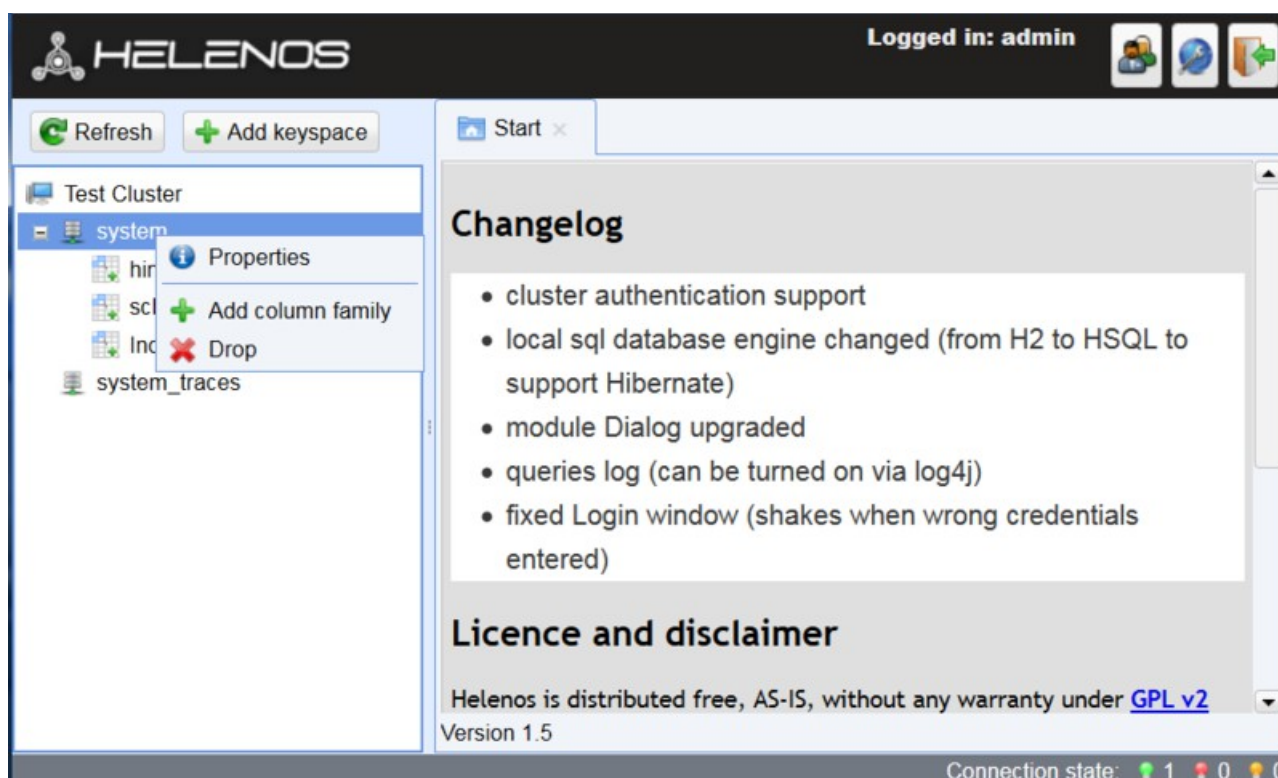


Figure III : interface Helenos

https://commons.wikimedia.org/wiki/File:Helenos_for_Apache_Cassandra.PNG (posté par moi-même)

Cassandra-Cluster-Admin

Les sources PHP sont sur <https://github.com/sebgiroux/Cassandra-Cluster-Admin>. A choisir si on a déjà son propre serveur Apache.

Elle est accessible sur <http://localhost/Cassandra-Cluster-Admin/> :

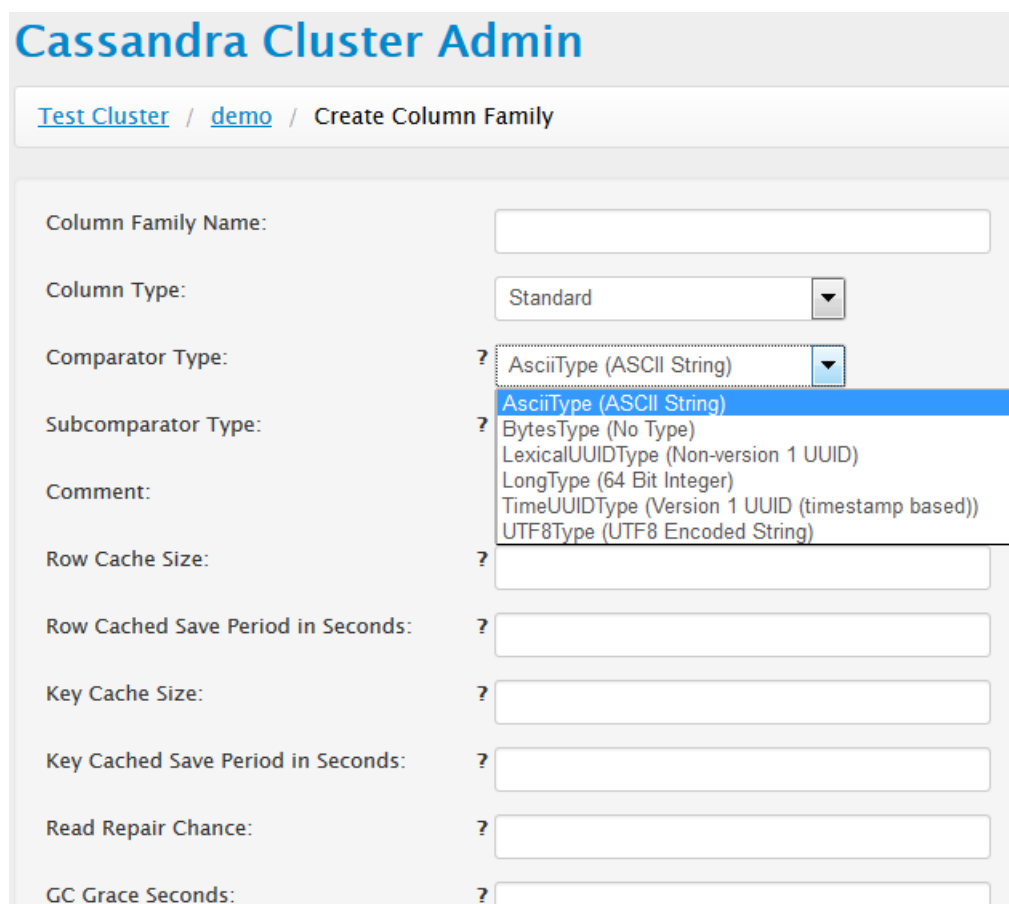


Figure III : interface Cassandra Cluster Admin

Screenshot privé, copyright © All Rights Reserved - Sébastien Giroux - 2011

Malheureusement aucune de ces deux interfaces ne permet d'exécuter le code CQL de créations de structures. En effet, elles imposent de créer chaque champ avec la souris.

Pour automatiser une création, il faut donc utiliser l'invite de commandes cqlsh.

De plus, elles ne voient pas les familles de colonnes créées avec les commandes ci-dessous via cqlsh (même si cqlsh voit les familles de colonnes créées par les interfaces).

OpsCenter

Ce logiciel maintenu par DataStax assure le monitoring et présente un tableau de bord sur <http://localhost:8888/>.

NB : il ne fonctionne pas sur mon Windows 7 64 bits.

Langage de manipulation de données

Code SQL de MySQL

```
CREATE DATABASE IF NOT EXISTS `banque-edel` DEFAULT CHARACTER SET utf8 COLLATE  
utf8_general_ci;  
USE `banque-edel`;
```

```
CREATE TABLE IF NOT EXISTS `Commandes` (  
  `id` int(11) NOT NULL auto_increment,  
  `RefCommande` int(11) NOT NULL,  
  `NoLigneCommandeTheorique` Int NOT NULL,  
  `DateCommande` Varchar(8) NOT NULL,  
  `TYPE_PREST` Varchar(2) NOT NULL,  
  `Destinataire` Varchar(32) default NULL,  
  `Complement` Varchar(32) default NULL,  
  `Adresse1` Varchar(32) default NULL,  
  `Adresse2` Varchar(32) default NULL,  
  `CodePostal` Varchar(5) default NULL,  
  `Ville` Varchar(32) default NULL,  
  `Pays` Varchar(15) default NULL,  
  `Telephone` Varchar(15) default NULL,  
  `Mail` Varchar(40) default NULL,  
  `Prefixe` Varchar(25) default NULL,  
  `Organisateur` Varchar(26) default NULL,  
  `DocumentAdditionnel` Varchar(5) default NULL,  
  `QuantiteTotale` Int NOT NULL,  
  `ValeurFacialeTotale` Varchar(20) default NULL,  
  `PND` boolean default 2,  
  `NbPND` Int default 0,  
  `NUM_EXP` Varchar(128) NOT NULL,  
  `TYPE_EXP` Int NOT NULL,  
  `DATE_EXP` Varchar(8) NOT NULL,  
  `date_modification` text NOT NULL,  
  PRIMARY KEY (`id`)  
) ENGINE=MyISAM DEFAULT CHARSET=utf8 AUTO_INCREMENT=1 ;
```

```
CREATE TABLE IF NOT EXISTS `LignesDeCommande` (  
  `id` int(11) NOT NULL auto_increment,  
  `id_MRBcommande` int(11) NOT NULL,  
  `NoLigneCommande` Int NOT NULL,  
  `TYPE_CARTE` Varchar(4) NOT NULL,  
  `Quantite` Int NOT NULL,  
  `ValeurFaciale` Varchar(20) default NULL,  
  `DateValidite` Varchar(8) default NULL,  
  `Perso1` Varchar(50) default NULL,  
  `Perso2` Varchar(50) default NULL,  
  `Perso3` Varchar(50) default NULL,  
  `Perso4` Varchar(50) default NULL,  
  `Perso5` Varchar(50) default NULL,  
  `Perso6` Varchar(50) default NULL,  
  `ID_LG_CMD` Int NOT NULL,  
  `NOM_CARTE` Varchar(50) default NULL,  
  `date_modification` text NOT NULL,  
  PRIMARY KEY (`id`)  
) ENGINE=MyISAM DEFAULT CHARSET=utf8 AUTO_INCREMENT=1 ;
```

Code CQL de Cassandra

<https://cassandra.apache.org/doc/cql3/CQL.html> donne une syntaxe CQL 3.0, utilisée pour convertir le code MySQL ci-dessus :

```
CREATE KEYSPACE BanqueEdel WITH REPLICATION = { 'class' : 'SimpleStrategy', 'replication_factor' :  
1 };  
USE BanqueEdel;
```

```
CREATE TABLE Commandes (  
id text,  
RefCommande text,  
NoLigneCommandeTheorique text,  
DateCommande text,  
TYPE_PREST text,  
Destinataire text,  
Complement text,  
Adresse1 text,  
Adresse2 text,  
CodePostal text,  
Ville text,  
Pays text,  
Telephone text,  
Mail text,  
Prefixe text,  
Organisateur text,  
DocumentAdditionnel text,  
QuantiteTotale text,  
ValeurFacialeTotale text,  
PND text,  
NbPND text,  
NUM_EXP text,  
TYPE_EXP text,  
DATE_EXP text,  
date_modification text,  
PRIMARY KEY(id));
```

Test :

```
SELECT * FROM system.schema_keyspaces;  
INSERT INTO Commandes (id, RefCommande) VALUES ('1', 'test');  
SELECT * FROM Commandes;  
DROP TABLE Commandes;
```

Principales différences de Cassandra par rapport à MySQL :

- 1) Le nom de l'espace de clés ne peut pas contenir de tiret ou de caractère Unicode (Invalid syntax).
- 2) Il n'y a pas de type VarChar de longueur personnalisable.
- 3) Pas d'option "autoincrement" des clés primaires, ou "not null".
- 4) Le mot TABLE peut être remplacé par COLUMNFAMILY.
- 5) La casse est ignorée.
- 6) Impossible de renommer un espace de clés.

ETL

Enfin pour importer la totalité des commandes passées, on utilise l'exécutable « sstableloader ».

Une telle importation de fichiers CSV ne doit pas dépasser 1 To, et nécessite de créer préalablement la structure via cqlsh.

- 1) Sous Ubuntu, le lancement de cqlsh renvoie "*dynamic module does not define init function (init_ssl)*".
- 2) Sous Windows, l'espace de clé « banqueedel » et sa table « commandes » ont pu être créés, mais les « sstableloader » a renvoyé les erreurs ci-dessous :

```
C:\Cassandra\bin>sstableloader -d 127.0.0.1 C:/banqueedel/
Could not retrieve endpoint ranges:
InvalidRequestException(why:No such keyspace: )
java.lang.RuntimeException: Could not retrieve endpoint ranges:
    at org.apache.cassandra.tools.BulkLoader$ExternalClient.init(BulkLoader.
java:338)
    at org.apache.cassandra.io.sstable.SSTableLoader.stream(SSTableLoader.ja
va:156)
    at org.apache.cassandra.tools.BulkLoader.main(BulkLoader.java:106)
Caused by: InvalidRequestException(why:No such keyspace: )
    at org.apache.cassandra.thrift.Cassandra$describe_ring_result$describe_r
ing_resultStandardScheme.read(Cassandra.java:35320)
    at org.apache.cassandra.thrift.Cassandra$describe_ring_result$describe_r
ing_resultStandardScheme.read(Cassandra.java:35287)
    at org.apache.cassandra.thrift.Cassandra$describe_ring_result.read(Cassa
ndra.java:35229)
    at org.apache.thrift.TServiceClient.receiveBase(TServiceClient.java:78)
    at org.apache.cassandra.thrift.Cassandra$Client.recv_describe_ring(Cassa
ndra.java:1292)
    at org.apache.cassandra.thrift.Cassandra$Client.describe_ring(Cassandra.
java:1279)
    at org.apache.cassandra.tools.BulkLoader$ExternalClient.init(BulkLoader.
java:304)
... 2 more
```

La littérature Internet ne permettant pas d'utiliser les différents ETL [ETL] pour l'instant [Erreurs], on choisira pour l'étude de convertir les CSV en commandes CQL.

Le plus simple serait donc d'utiliser la commande « copy », toutefois elle s'avère inopérante dans notre configuration :

```
cqlsh:banqueedel> copy Commandes (id,RefCommande,NoLigneCommandeTheorique,DateCommande,TYPE_PREST,Destinataire,Complement,Adresse1,Adresse2,CodePostal,Ville,Pays,Telephone,Mail,Prefixe,Organisateur,DocumentAdditionnel,QuantiteTotale,ValeurFacialeTotale,PND,NbPND,NUM_EXP,TYPE_EXP,DATE_EXP,date_modification) from 'banqueedel.csv';
```

Error starting import process:

*Can't pickle <type 'thread.lock'>: it's not found as thread.lock
can only join a started process*

```
cqlsh:banqueedel> Traceback (most recent call last):
```

File "<string>", line 1, in <module>

*File "C:\Python27\lib\multiprocessing\forking.py", line 380, in main
prepare(preparation_data)*

*File "C:\Python27\lib\multiprocessing\forking.py", line 489, in prepare
file, path_name, etc = imp.find_module(main_name, dirs)*

ImportError: No module named cqlsh

API

Pour automatiser le processus d'insertion, on cherche à utiliser l'API Cassandra (<https://wiki.apache.org/cassandra/API>) au moyen d'un pilote contenant des classes. Certains sont disponibles sur <http://www.datastax.com/download-drivers> pour le Java, C/C++, C#, JavaScript (Node.js), Python, Ruby, PHP (bêta) et Spark.

Le site actuel pour Banque-Edel étant en PHP, le pilote doit être ajouté dans php.ini :

```
extension=pdo_cassandra.so
```

Or, cette extension PDO n'est pas disponible au téléchargement et doit être construite à partir du code source. Sous Windows cela nécessite d'utiliser le logiciel Cmake.

Error: The source directory does not appear to contain CMakeLists.txt.

En téléchargeant ce .txt du pilote CPP de GitHub :

Configuring incomplete, errors occurred!

Pour les besoins de l'étude, on utilisera donc l'invite de commande cqlsh.

Code CQL d'insertion

On regroupe les deux tables MySQL en une seule correspondant exactement aux CSV, en s'assurant que les champs sont compatibles UTF8 :

```
CREATE COLUMNFAMILY Commandes (id text,RefCommande text,NoLigneCommande
text,NoLigneCommandeTheorique text,DateCommande text,TYPE_PREST text,Destinataire
text,Complement text,Adresse1 text,Adresse2 text,CodePostal text,Ville text,Pays text,Telephone text,Mail
text,Prefixe text,Organisateur text,DocumentAdditionnel text,TYPE_CARTE text,Quantite text,ValeurFaciale
text,DateValidite text,Perso1 text,Perso2 text,Perso3 text,Perso4 text,Perso5 text,Perso6 text,ID_LG_CMD
text,NOM_CARTE text,PND text,NbPND text,NUM_EXP text,TYPE_EXP text,DATE_EXP
text,date_modification text, PRIMARY KEY(id));
```

Puis on copie les commandes d'insertion obtenues à partir du script ci-dessous qui affiche les contenus des fichiers CSV déjà importés dans MySQL :

```
<?php
$rep = 'C:\Users\hbossot\Documents\www\banque-edel\www\ressources\BDD\Faits';
$dir = opendir($rep);
$cid = 0;
while($f = readdir($dir)) {
    if (!is_dir($f)) {
        $ACK = ConversionCQL($f);
    }
}
closedir($dir);

function ConversionCQL($f) {
    global $rep;
    global $cid;
    $NomCSV = $rep.'\\'.$f;
    $f = fopen($NomCSV, 'r');
    if ($f) {
        $Commandes = array(0=>'RefCommande', 1=>'NoLigneCommande',
2=>'NoLigneCommandeTheorique', 3=>'DateCommande', 4=>'TYPE_PREST',
5=>'Destinataire', 6=>'Complement', 7=>'Adresse1', 8=>'Adresse2', 9=>'CodePostal',
10=>'Ville', 11=>'Pays', 12=>'Telephone', 13=>'Mail', 14=>'Prefixe', 15=>'Organisateur',
16=>'DocumentAdditionnel', 17=>'TYPE_CARTE', 18=>'Quantite', 19=>'ValeurFaciale',
20=>'DateValidite', 21=>'Perso1', 22=>'Perso2', 23=>'Perso3', 24=>'Perso4',
25=>'Perso5', 26=>'Perso6', 27=>'ID_LG_CMD', 28=>'NOM_CARTE');
        while (($ligne = fgets($f, 4096)) !== false) {
            print 'INSERT INTO Commandes (id, '.str_replace("'", "", implode(",",
$Commandes)).') VALUES ("'. $cid.'", "'.str_replace(";", "\",', $ligne).'\");<br/>';
            $cid++;
        }
        fclose($f);
    }
}
?>
```

L'importation des 9 051 lignes de commandes dure cinq minutes et se traduit sous la forme d'un rafraîchissement du nombre d'opérations sur la table « commandes », dans la console de lancement du serveur Cassandra :

```
INFO 20:02:14 ColumnFamily Memtable ops,data
INFO 20:02:14 system.schema_triggers 0,0
INFO 20:02:14 system.local 4,450
INFO 20:02:14 system.peers 0,0
INFO 20:02:14 system.batchlog 0,0
INFO 20:02:14 system.size_estimates 0,0
INFO 20:02:14 system.schema_usertypes 0,0
INFO 20:02:14 system.compactions_in_progress 0,0
INFO 20:02:14 system.schema_keyspaces 0,0
INFO 20:02:14 system.compaction_history 9,1199
INFO 20:02:14 system.paxos 0,0
INFO 20:02:14 system.schema_columns 0,0
INFO 20:02:14 system.schema_columnfamilies 0,0
INFO 20:02:14 system.IndexInfo 0,0
INFO 20:02:14 system.range_xfers 0,0
INFO 20:02:15 system.peer_events 0,0
INFO 20:02:15 system.hints 0,0
INFO 20:02:15 system.sstable_activity 4,736
INFO 20:02:15 banqueedel.commandes 6660,956704
INFO 20:02:15 system_traces.sessions 0,0
INFO 20:02:15 system_traces.events 0,0
```

```
INFO 20:07:12 ColumnFamily Memtable ops,data
INFO 20:07:12 system.schema_triggers 0,0
INFO 20:07:12 system.local 4,450
INFO 20:07:12 system.peers 0,0
INFO 20:07:12 system.batchlog 0,0
INFO 20:07:12 system.size_estimates 2310,1054033
INFO 20:07:12 system.schema_usertypes 0,0
INFO 20:07:12 system.compactions_in_progress 0,0
INFO 20:07:12 system.schema_keyspaces 0,0
INFO 20:07:12 system.compaction_history 9,1199
INFO 20:07:12 system.paxos 0,0
INFO 20:07:12 system.schema_columns 0,0
INFO 20:07:12 system.schema_columnfamilies 0,0
INFO 20:07:12 system.IndexInfo 0,0
INFO 20:07:12 system.range_xfers 0,0
INFO 20:07:12 system.peer_events 0,0
INFO 20:07:12 system.hints 0,0
INFO 20:07:12 system.sstable_activity 4,736
INFO 20:07:12 banqueedel.commandes 39360,5676836
INFO 20:07:12 system_traces.sessions 0,0
INFO 20:07:12 system_traces.events 0,0
```

On remarquera au passage qu'insérer deux enregistrements avec la même clé primaire efface le premier sans avertissement.

Performances

Chaque nœud doit posséder dans sa configuration au moins un « seed node » pour lui apprendre l'architecture du système distribué [DigitalOcean], appelé « anneau » (ring). Leur localisation physique est spécifiable dans `\cassandra\conf\cassandra-rackdc.properties`.

La scalabilité est censée être linéaire, deux nœuds traitent deux fois plus de transactions qu'un seul [DataStax].

Chaque nœud possède une balance (snitch) correspondant au pourcentage des bases possédés. A partir du moment où le paramètre de `cassandra.yaml` « `endpoint_snitch` » est modifié de *SimpleSnitch* à *RackInferringSnitch*, les nœuds en réseau se connectent entre eux.

Pour lister les nœuds répliqués, la commande est ci-dessous (192.168.1.2 est le seed) :

```
>nodetool status
Starting NodeTool
Datacenter: 168
=====
Status=Up/Down
// State=Normal/Leaving/Joining/Moving
-- Address    Load    Tokens  Owns  Host ID  Rack
UN 192.168.1.3 244,89 KB 256    ?    340f7993-e3b2-44e9-b33d-7064fb32b933 1
UN 192.168.1.2 201,64 KB 256    ?    52cd0c60-c981-4deb-9f2e-98602c034c19 1
```

NB : il faut plusieurs minutes après une modification de table pour que la colonne « Owns » se stabilise. Mais en créant un espace de clés similaire mais avec un « `'replication_factor' : 1` », La colonnes « Owns » est toujours à 100 :

```
>nodetool status
Starting NodeTool
Datacenter: 168
=====
Status=Up/Down
// State=Normal/Leaving/Joining/Moving
-- Address    Load    Tokens  Owns (effective)  Host ID  Rack
UN 192.168.1.3 143,7 KB 256    100,0%            340f7993-e3b2-44e9-b33d-7064fb32b933 1
UN 192.168.1.2 218,64 KB 256    100,0%            52cd0c60-c981-4deb-9f2e-98602c034c19 1
```

Les données sont adressées par des tokens, résultats d'une fonction de hachage.

Aucun test de répartition manuelle entre les nœuds n'a pu être effectué, car changer le paramètre "initial_node" du n° 2 l'empêche de se lancer.

Par ailleurs, il est possible de configurer deux type de répartitions automatiques [Apache] :

- *RackUnawareStrategy* : les réplicas sont toujours placés sur le prochain nœud de l'anneau.
- *RackAwareStrategy* : chaque réplica est placé sur le premier noeud de l'anneau appartenant à un autre datacenter que son prédécesseur.

La répartition de chacun des 256 tokens du ring elle est vue par Cassandra Cluster Admin :

Keyspace Details

Strategy Class: org.apache.cassandra.locator.SimpleStrategy
Strategy Options: None
Replication Factor: 2

Ring

- Start Token: 6497303436854464065
- End Token: 6531185686014574864
- Endpoints: 192.168.1.2
- Endpoints: 192.168.1.3
- Start Token: 4781986147697852947
- End Token: 4828713329197027019
- Endpoints: 192.168.1.2
- Endpoints: 192.168.1.3
- ...
- Start Token: -381790296957924542
- End Token: -376185223093445408
- Endpoints: 192.168.1.2
- Endpoints: 192.168.1.3

Quand on arrête brutalement le nœud 2, le nombre d'enregistrement diminue de moitié :

```
cqlsh:banqueedel> select count(*) from commandes;
count
-----
2129
(1 rows)
```

A partir de cet état, en repassant sur un facteur de réplication unitaire, Cassandra retrouve la deuxième moitié instantanément :

```
ALTER KEYSPACE banqueedel WITH replication = {'class': 'SimpleStrategy', 'replication_factor' : 1};
SELECT COUNT(*) FROM commandes;
count
-----
4241
(1 rows)
```

Keyspace Details

Strategy Class: org.apache.cassandra.locator.SimpleStrategy
Strategy Options: None
Replication Factor: 1

Ring

- Start Token: 6497303436854464065
- End Token: 6531185686014574864
- Endpoints: 192.168.1.2
- Start Token: 4781986147697852947
- End Token: 4828713329197027019
- Endpoints: 192.168.1.2
- Start Token: 5111031796484994446
- End Token: 5125728175126093178
- Endpoints: 192.168.1.2
- Start Token: -4832848052647861836
- End Token: -4817377274624019579
- Endpoints: 192.168.1.2
- Start Token: -8208376777850896573
- End Token: -8164086156485998530
- Endpoints: 192.168.1.2
- Start Token: -508924529456650901
- End Token: -504309206211009296
- **Endpoints: 192.168.1.3**
- ...
- Start Token: -7621373298032027535
- End Token: -7592149456842269770
- **Endpoints: 192.168.1.3**
- Start Token: -8276327345755685286
- End Token: -8264839959329320081
- Endpoints: 192.168.1.2
- Start Token: -381790296957924542
- End Token: -376185223093445408
- Endpoints: 192.168.1.2

Toutefois, le seed attribue quand-même des tokens au nœud déconnecté (192.168.1.3).

Conclusion

En somme, le SGBD orienté colonne Apache Cassandra ne possède pas encore l'ergonomie et la comptabilité des principaux SGBDR.

En effet, il n'a pas permis de mettre en place un système automatique via son pilote PHP encore en développement, et la console CQLSH sature aléatoirement quand on y insère des milliers de commandes manuellement.

Toutefois ses performances d'accès et de répartition automatique se sont révélées prometteuses, et il sera intéressant de retester avec un site plus petit mais très fortement visité pour en bénéficier pleinement [mbaron].

Webographie

[ETL]

<http://www.datastax.com/dev/blog/bulk-loading>

<https://github.com/mbaird/sstableloader-csv>

<https://github.com/brianmhess/cassandra-loader>

<https://github.com/datastax/php-driver>

[Erreurs]

www.9threes.com/2014/07/bulk-loading-data-into-cassandra-using.html

<http://stackoverflow.com/questions/30627309/pickling-error-running-copy-command-cqlshell-on-windows/>

[DataStax]

http://docs.datastax.com/en/getting_started/doc/getting_started/gettingStartedCassandraIntro.html

[DigitalOcean]

<https://www.digitalocean.com/community/tutorials/how-to-configure-a-multi-node-cluster-with-cassandra-on-a-ubuntu-vps>

[Apache]

<https://wiki.apache.org/cassandra/Operations>

[mbaron]

<http://mbaron.developpez.com/tutoriels/nosql/cassandra/installation-outils-administration/>